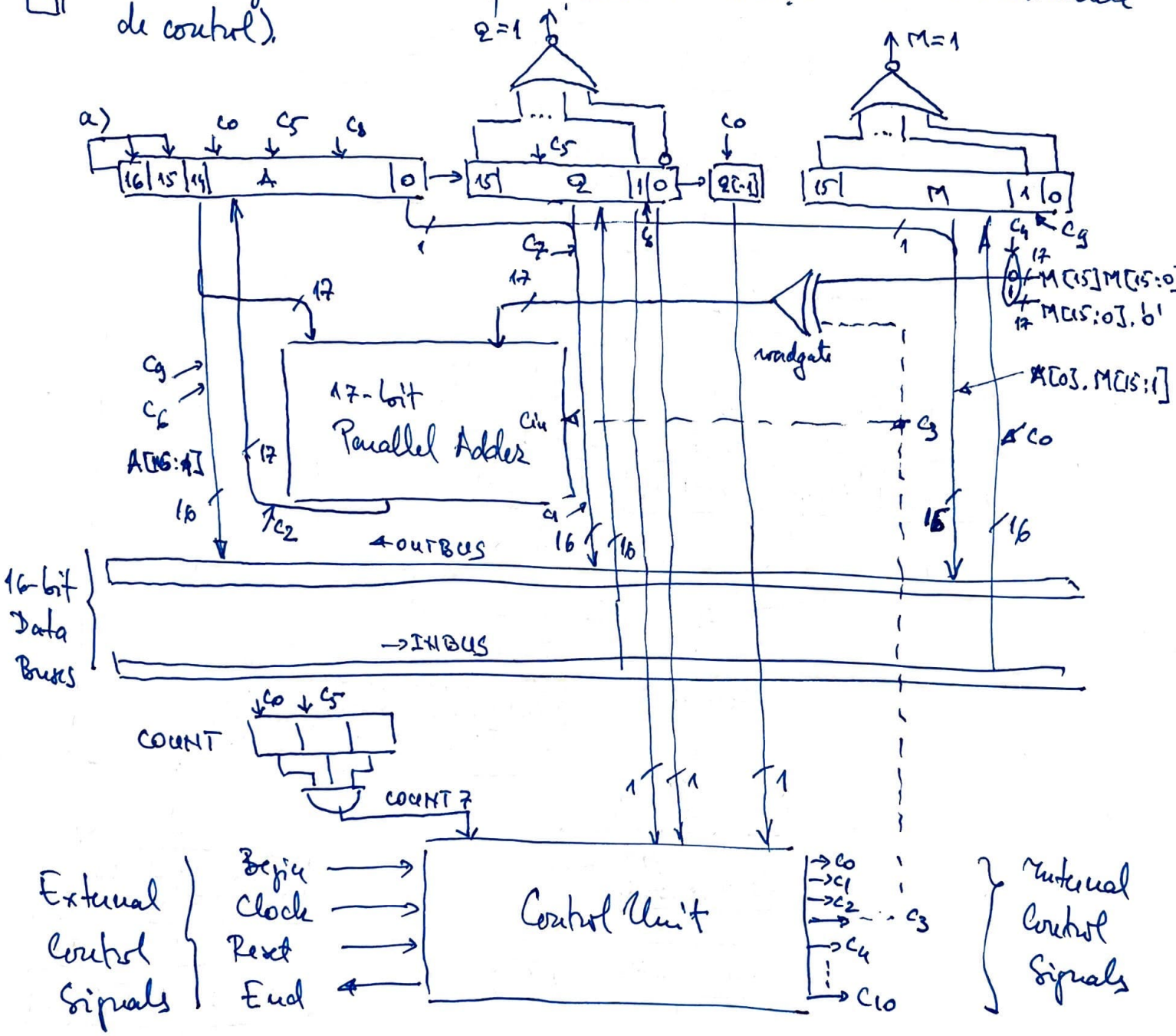
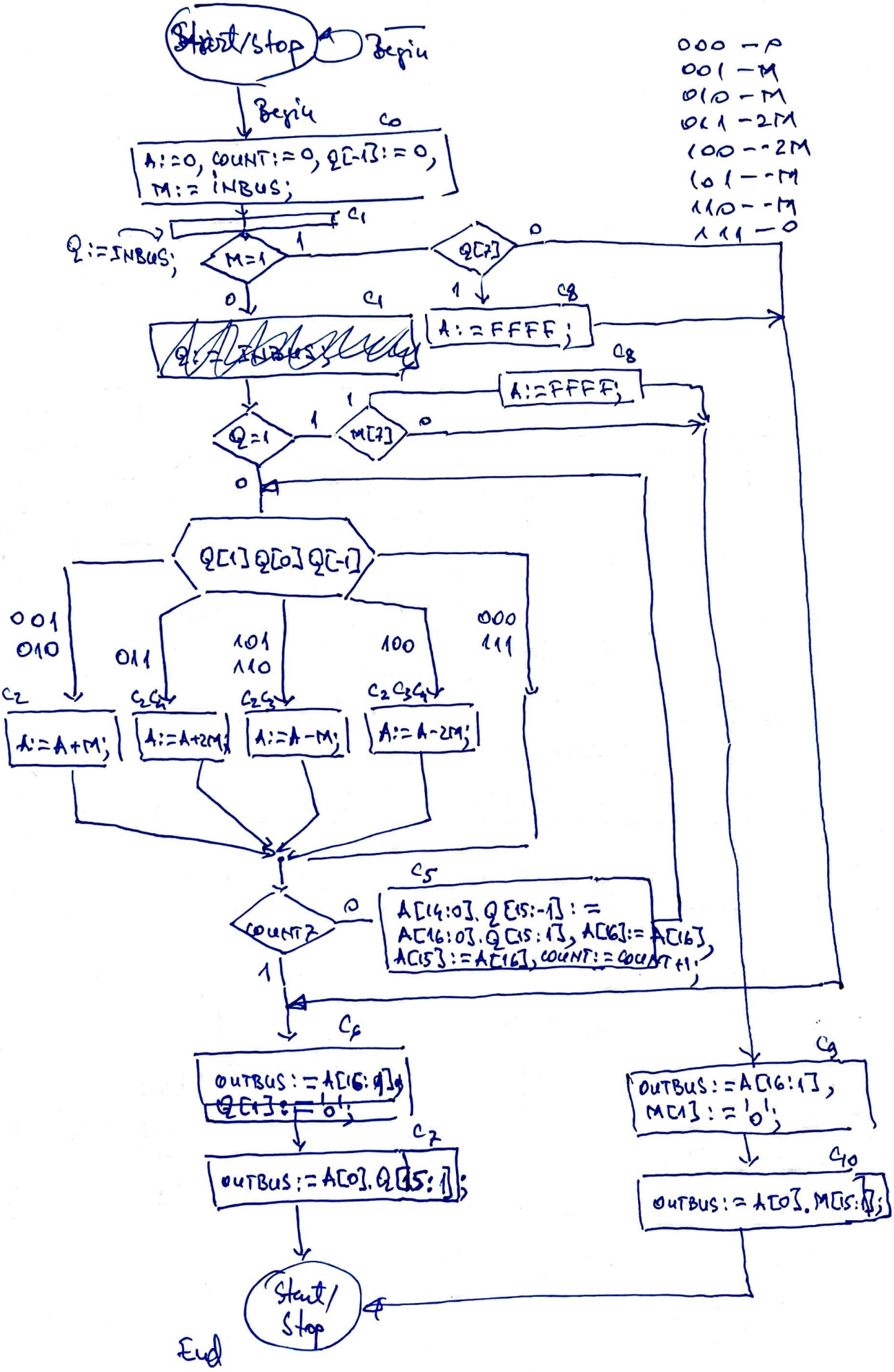
(2p) (1) Să se modifice dispozitivul de înmulțire Booth radix-4, astfel încât acesta să înmulțească numere fracționare reprez. pe 16 biți și să trateze în mod distinct situațiile de înmulțire cu unu. (Rezultatul este egal cu celălalt operand, exprimat pe 32 biți.) Se cer:

(1p) a) Schema bloc detaliată pe platforma ~~Leadtronic~~ Hayes. (cu semnale de ctrl.)

(1p) b) Ordinograma ce descrie funcționarea unității de control (cu semnalele de control).



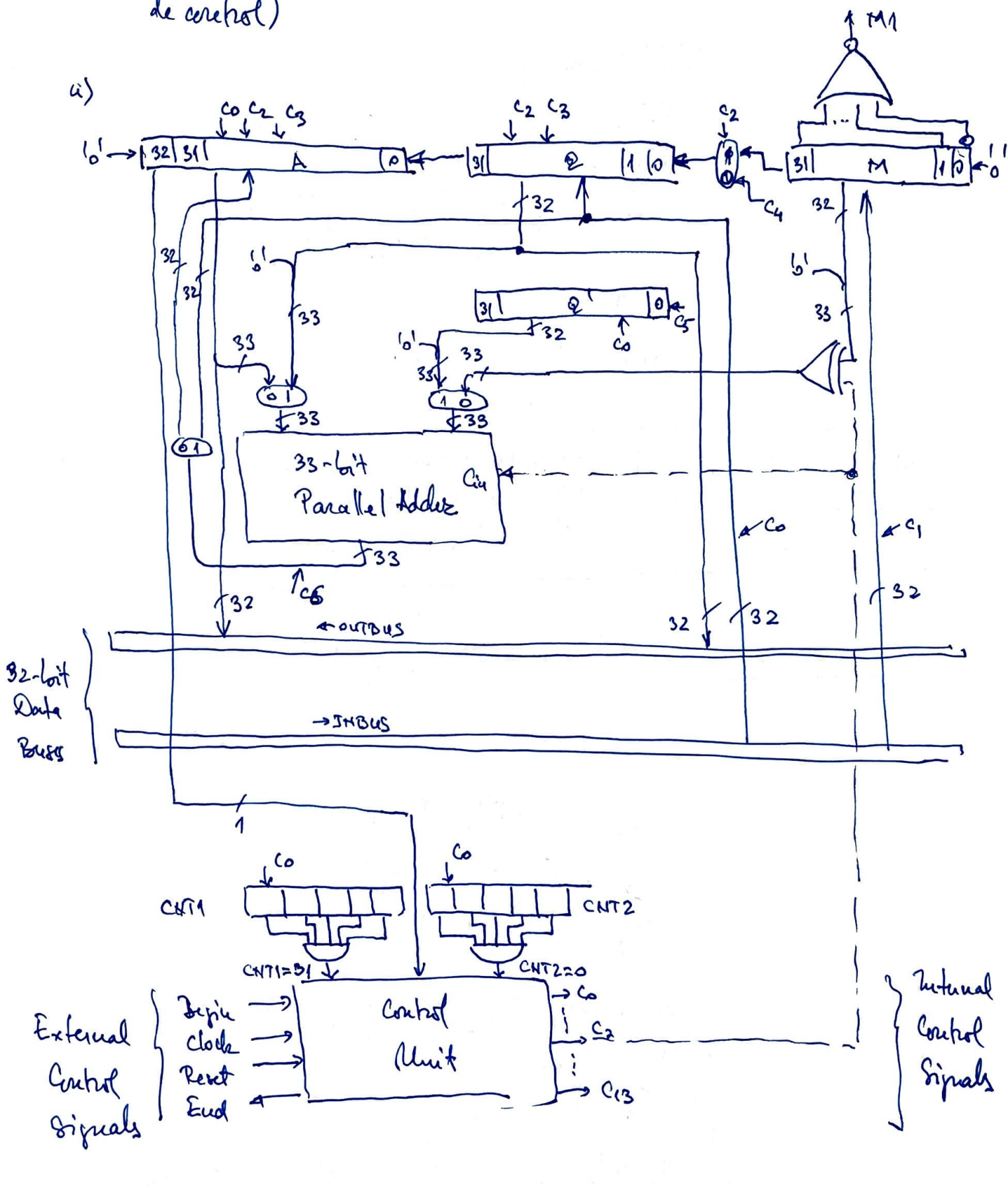

Start/stop
Begin
Begin
c0
A:=0, COUNT:=0, Q[-1]:=0,
M:=INBUS;
c1
Q:=INBUS;
M=1
1
0
Q[7]
0
1
c1
c8
A:=FFFF;
A:=FFFF;
c8
Q=1
1
M[7]
1
0
0
Q[1] Q[0] Q[-1]
001
010
011
101
110
100
000
111
c2
c2c4
c2c3
c2c3c4
A:=A+M;
A:=A+2M;
A:=A-M;
A:=A-2M;
c5
A[14:0], Q[15:-1] :=
A[16:0], Q[15:1], A[6]:=A[16],
A[5]:=A[16], COUNT:=COUNT+1;
COUNT7
0
1
c6
OUTBUS:=A[16:9];
Q[1]:='0';
c7
OUTBUS:=A[0], Q[15:1];
c9
OUTBUS:=A[16:1],
M[1]:='0';
c10
OUTBUS:=A[0], M[15:1];
Start/
Stop
End
000 – P
001 – M
010 – M
011 – 2M
100 – -2M
101 – -M
110 – -M
111 – 0

① Să se modifice dispozitivul de împărțire SRT radix-2 astfel încât acesta
② să împartă numere pe 32 de biți fără a genera cifra câtului 0 (doar 1 și $\bar{1}$) și să trateze în mod distinct situația de împărțire la 1. Se cer:

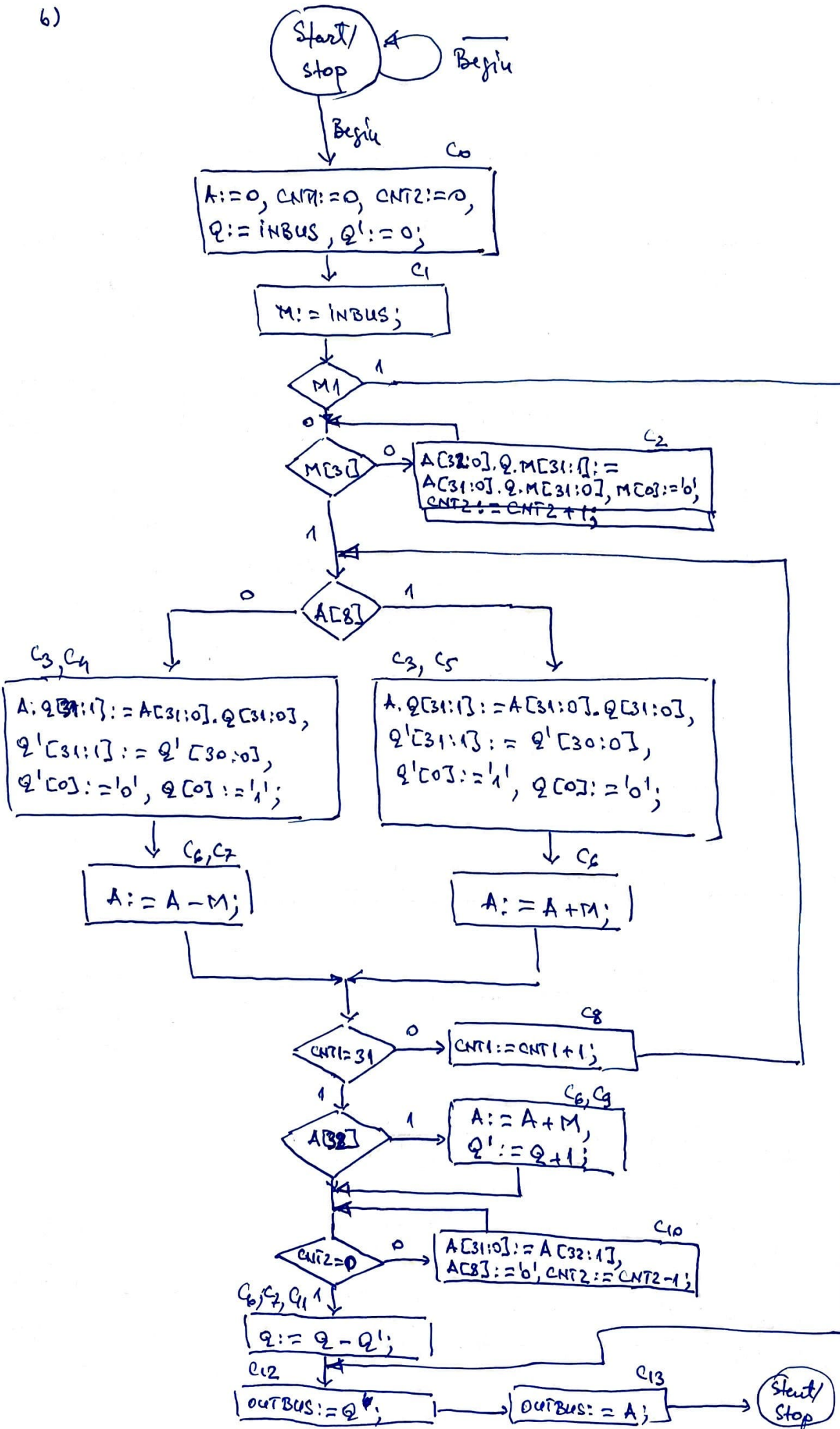
a) Schema bloc hardware folosind platforma Hayes + semnalele de control

b) Ordinograma ce descrie funcționarea unității de control (+ semnalele de control)

a)



6)


Start/ stop
Begin
Begin
C0
A:=0, CNT1:=0, CNT2:=0,
Q:=INBUS, Q':=0;
C1
M:=INBUS;
M1
1
0
M[3]
0
C2
A[32:0], Q.M[31:1]:=
A[31:0].Q.M[31:0], M[0]:='0';
CNT2:=CNT2+1;
1
A[8]
0
1
C3, C4
A.Q[31:1]:=A[31:0].Q[31:0],
Q'[31:1]:=Q'[30:0],
Q'[0]:='0', Q[0]:='1';
C3, C5
A.Q[31:1]:=A[31:0].Q[31:0],
Q'[31:1]:=Q'[30:0],
Q'[0]:='1', Q[0]:='0';
C6, C7
A:=A-M;
C6
A:=A+M;
C8
CNT1=31
0
CNT1:=CNT1+1;
1
C6, C9
A[32]
1
A:=A+M,
Q':=Q+1;
C10
CNT2=0
0
A[31:0]:=A[32:1],
A[8]:='0', CNT2:=CNT2-1;
C6, C7, C11
1
Q:=Q-Q';
C12
OUTBUS:=Q';
C13
OUTBUS:=A;
Start/ Stop

(2) (2p) Considerăm o mașină de calcul X caracterizată printr-un clock frequency de 5 GHz, care rulează un benchmark suite ce generează următoarea statistică pe tipuri de instrucțiuni.

Instr. type	Clock cycles	Instruction Count
AL	5	5000
MOV	3	10000
L/S	3	3000
Other	2	2000

Restul rămân nemodificate

Această mașină este optimizată de 2 echipe, una de hardware, alta de software. Echipa hardware optimizează implementarea setului de instrucțiuni, reducând numărul de cicluri de clock pretendicate de instrucțiunile AL de la 5 la 3 și de cele L/S de la 3 la 2. (Această optimizare are drept consecință degradarea ~~clock frequency~~ cycle time la nivel de instruction set cu 5%.)

Echipa software oferă o optimizare de compilator, ce reduce numărul de instrucțiuni MOV ~~și~~ cu 20% și cel de instrucțiuni Other cu 10%.

Să se calculeze care echipă - cea HW sau cea SW - a făcut o treabă mai bună, în conformitate cu

a) CPU time (1p) b) MIPS (1p)

a) $\text{Clock cycle time}_X = \frac{1}{5\,\text{GHz}} = \frac{1}{5 \cdot 10^9}\,\text{s} = 0.2\,\text{ns}$

$\text{CPU time}_X = (25000 + 30000 + 9000 + 4000) \times 0.2\,\text{ns} =$

$= 68000 \times 0.2\,\text{ns} = 13600\,\text{ns} = \boxed{13.6\,\mu\text{s}}$

~~Clock cycle time~~ frequency $_{HW} = 5\,\text{GHz} - 0.05 \times 5\,\text{GHz} = 0.95\,\text{GHz}$

~~Clock cycle time~~ $_{HW} = \frac{1}{0.95 \times 10^9}\,\text{s} =$

$\text{Clock cycle time}_{HW} = 0.2\,\text{ns} \times 1.05 = 0.21\,\text{ns}$

$\text{CPU time}_{HW} = (3 \times 5000 + 3 \times 10000 + 2 \times 3000 + 2 \times 2000) \times 0.21\,\text{ns}$

$= 55000 \times 0.21\,\text{ns} = 11550 = \boxed{11.55\,\mu\text{s}}$

CPU time SW

$IC_{AL_{SW}} = 5000$

$IC_{MOV_{SW}} = 8000$

$IC_{LS_{SW}} = 3000$

$IC_{other_{SW}} = 1800$

$CPU\,time_{SW} = (5000 \times 5 + 8000 \times 3 + 3000 \times 3 + 1800 \times 2) \times 0.2\,\mu s =$

$= 61\,600 \times 0.2\,\mu s = 12\,320\,\mu s$

$= \boxed{12.320\,\mu s}$

=> HW team is better!

b) $IC_{HW} = 20000$

$IC_{SW} = 17800$

$MIPS_{HW} = \frac{20000}{12.320\,\mu s \times 10^6} =$

$= \frac{20\,000}{11\,550 \times 10^{-9} \times 10^6} = \frac{20000}{11550} \times 10^3$

$= 17316 \times 10^3 = \boxed{1731.6}$

$MIPS_{SW} = \frac{17800}{12\,320 \times 10^{-9} \times 10^6} = \boxed{1444,8}$

(12) (2p) Considerăm o mașină X caracterizată printr-un clock frequency de 4.5 GHz, care rulează un benchmark suite ce generează următoarea statistică pe tipuri de instrucțiuni

Instr. type	Clock cycles	Instruction Count
AL	4	10000
MOV	3	10000
L/S	2	7000
Other	1	3000

Această mașină este optimizată de 2 echipe, una HW și alta SW. Echipa HW optimizează implementarea setului de instrucțiuni, reducând numărul de cicli de clock revendicați de instrucțiunile AL de la 4 la 3 și MOV de la 3 la 2. Restul rămân neschimbate. Această optimizare ~~a~~ a fost dublată de alimentarea CPU-ului la o tensiune crescută și implicit de o îmbunătățire a clock cycle time cu 5%.

Echipa SW oferă o optimizare de compilator, ce reduce numărul de instrucțiuni AL și MOV cu 15% și cel de L/S cu 10%.

Să se calculeze care echipă - cea HW sau cea SW - a făcut o treabă mai bună, în conformitate cu

a) CPU time (1p)

b) MIPS (1p)

---

a) Clock cycle time$_X$ = $\frac{1}{4.5\,GHz} = \frac{1}{4.5 \cdot 10^9}\,s = 0.2222\,ns$ =

= Clock cycle time$_{SW}$

CPU time$_X$ = $(4 \times 10000 + 3 \times 10000 + 2 \times 7000 + 1 \times 3000) \times 0.2222$

= $87000 \times 0.2222 = 19331.4\,ns = \boxed{19.3314\,\mu s}$

Clock cycle time$_{HW}$ = $0.95 \times 0.2222\,ns = \underline{0.211\,ns}$

$$CPU\,time_{HW} = (3 \times 10000 + 2 \times 10000 + 2 \times 7000 + 1 \times 3000) \times 0.211\,\mu s$$

$$= 14\,137\,\mu s = \boxed{14.137\,\mu s}$$

$IC_{ALSW} = 0.85 \times 10000 = 8500$

$IC_{MOVSW} = 0.85 \times 10000 = 8500$

$IC_{LSSW} = 0.9 \times 7000 = 6300$

$IC_{offlineSW} = 3000 = 3000$

$CPU\,time_{SW} = (7 \times 8500 + 2 \times 6300 + 3000) \times 0.2222\,\mu s$

$= 16\,687.22\,\mu s$

$= \boxed{16.68722\,\mu s}$

b) $IC_{HW} = 30000$

$IC_{SW} = 26\,300$

$$MIPS_{HW} = \frac{30\,000}{14.137 \cdot 10^{-6} \times 10^{6}} = 2122.09$$

$$MIPS_{SW} = \frac{26\,300}{16.68722 \cdot 10^{-6} \cdot 10^{6}} = 1576.0564$$

(3) (2p) Avem un computer load/store cu un sistem cache ce se caracterizează prin următoarele valori empirice atunci când rulăm un anumit program: 10% Data Miss Rate, 8% Instruction Miss Rate și 20% instrucțiuni load/store. În orice moment, 20% din blocuri sunt modificate (i.e., dirty). Programul determină un CPI de 2 clock cycles dacă cache-ul este perfect și constă din 10 000 de instrucțiuni în cod mașină. Calculatorul are o frecvență a clock-ului de 4.5 GHz. Penalitatea de miss e determinată de faptul că un access la BUS ~~necesită~~ necesită 10 cicluri de clock, lățimea BUS-ului este de 4 cuvinte, block-ul are 16 cuvinte, iar cache-ul implementează o politică Write Back cu Write Allocate în cazul unui miss. Calculați:

(1p) a) ~~TMAT~~, știind că t_{ac} = 2 c.c.

(1p) b) CPU time.

---

a)

Read Miss Penalty = $\underbrace{\frac{16 \text{ words}}{4 \text{ words}} \times 0.2 \text{ BUS units}}_{\text{Update}} + \underbrace{\frac{16 \text{ words}}{4 \text{ words}} \text{ BUS Reads}}_{\text{Allocate}}$

= Write Miss Penalty

$= 4 \cdot 0.2 + 4 = 4(1.2) = 4.8$ BUS accesses

$\Rightarrow$ Miss Penalty = $4.8 \times 10 = \boxed{48 \text{ c.c.}}$

0.08
0.02
0.10

Clock cycle time = $\frac{1}{4.5 \cdot 10^9 s^{-1}} = 0.2222$ ns

0.2 4/5

1.(2)

Misses Per Instruction = Miss Rate × Mem. accesses per instruction

Misses Per Instruction = $1 \times IMR + 0.2 \times DMR = 0.08 + 0.2 \times 0.1$

Miss Rate = $\frac{0.1}{1.2} = 0.0833$ $= 0.1$

AMAT = 0.4444 us + 0.0833 × 48 cc × 0.2222 =

= **1.3328 us**

b) $CPU_{time} = 10000 \underbrace{(2 + 0.1 \times 48)}_{6.8} \times 0.2222 =$

$= 15109.6 \text{ us} = \mathbf{15.1096 \mu s}$

(3) (2p) Avem un computer L/S cu un sistem cache cu următoarele valori empirice atunci când rulăm un anumit program: 9% Data Miss Rate, 8% Instruction Miss Rate și 10% instrucțiuni L/S. În orice moment 10% din blocuri sunt modificate (i.e., dirty). Programul determină un CPI de 3 c.c. dacă cache-ul este perfect și constă din 1000 de instrucțiuni în cod mașină. Calculatorul are o frecvență a clock-ului de 5 GHz. Penalitatea de miss e determinată de faptul că un access la BUS reclamă 8 cicluri de clock, lățimea BUS-ului este de 2 cuvinte, block-ul are 32 de cuvinte iar cache-ul implementează o politică Write Through cu Write No Allocate în cazul unui miss.

10% ~~sunt~~ din acces sunt scrieri, restul sunt citiri.

Calculați:

(1p) a) AMAT, știind că $t_{ac} = 1$ c.c.

(1p) b) CPU time

---

a) Clock cycle time $= \frac{1}{5 \cdot 10^9 \, s^{-1}} = \boxed{0.2 \text{ ns}}$

Read Miss Penalty $= \frac{32 \text{ words}}{2 \text{ words}} = 16$ BUS reads
(Allocate)

Write Miss Penalty $= 1$ BUS write

Miss Penalty $= 0.1 \cdot$ Write Miss Penalty $+ 0.9 \cdot$ Read Miss Penalty
$= 0.1 \times 1 + 0.9 \times 16 = 14.5$ BUS accesses
$= 14.5 \times 8 = \boxed{116}$ clock cycles

Misses Per Instruction $=$ Miss Rate $\times$ (Mem acc per instr. $= 1.1$)
Misses Per Instr $= 0.08 + 0.09 \times 0.1 = 0.089$

$$\text{Miss Rate} = \frac{0.089}{1.1} = 0.0809$$

a) $AMAT = 0.2 + 0.0809 \times 116 \times 0.2 = \boxed{2.07688 \text{ us}}$

b) $CPU_{time} = 1000 \times (3 + 0.089 \times 116) \times 0.2 =$

$= 2664.8 \text{ us} = \boxed{2.664 \text{ ms}}$

(4) (3p) Se dă un sistem de memorie, cu spațiul virtual de dimensiune 256 TiB, spațiu fizic de 256 GiB și adresare la nivel de byte. Cuvântul este pe 64 de biți, block-ul are 64 de cuvinte. O pagină de memorie are dimensiunea de 64 KiB, iar dimensiunea de date a cache-ului (SA pe 2 căi) este de 256 KiB. Cache-ul este Write Back, cu Write Allocate în caz de Write Miss și are o politică de înlocuire random. TLB-ul are 64 de intrări și folosește mapare directă. Se cer:

(1p) a) Formatul adresei de memorie virtuală respectiv fizică

(1p) b) Dimensiunea totală a memoriei cache și a TLB-ului.

(1p) c) Schema bloc a mapării adresă fizică - adresă virtuală ce conține memoria cache și TLB-ul (ca în curs).

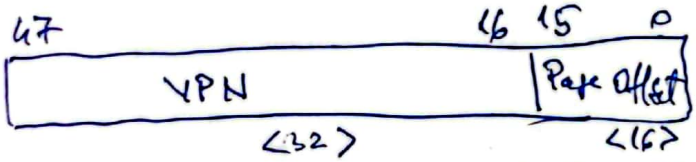
a) $VM_{size} = 2^8 \times 2^{40} B = 2^{48} B$ $\quad Page = 64 KiB = 2^{16} B$

$PM_{size} = 2^{38} B$

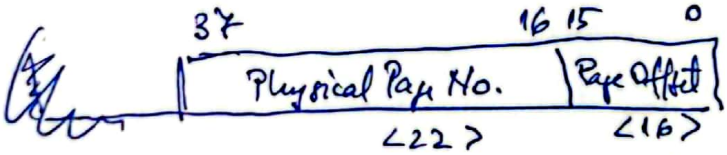
$1 word = 2^3 B$ → Word Offset

$1 block = 2^6 words = 2^6 \cdot 2^3 B = 2^9 B$ → Block Offset

$TLB\ entries = 2^6$ → TLB Index

✓ 

Virtual Memory

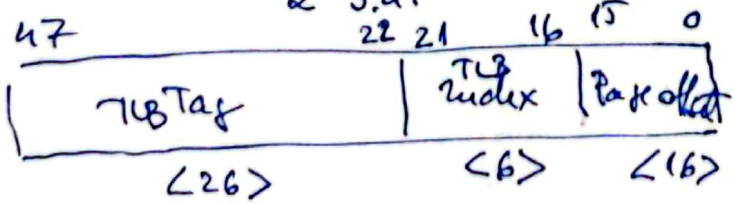
✓ 

Physical Memory

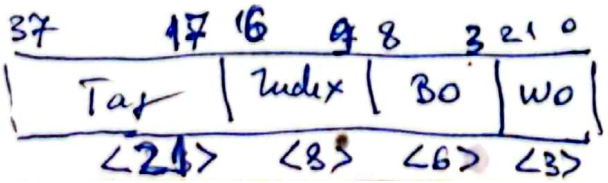
38 - 16 = 22

$Cache\ data\ size = 256 KiB = 2^{18} B = \frac{2^{18} B}{2^9 B/block} = 2^9 blocks$

$Bank\ size = \frac{2^9 blocks}{2^1 S.A.} = 2^8$ → Index $blocks/bank.$

✓ 

Virt Mem

✓ 

Phys. Mem

Scanned with CamScanner

b) $CacheTSize = \underbrace{2^1}_{s.a} \times \underbrace{2^8}_{index} \times (\underbrace{2^9 B}_{data\,block} + \underbrace{16}_{V} + \underbrace{216}_{Tag}) = 2^9 \times 2^9 B + 2^9 \cdot 3B =$ (216 dirty)

$= 2^{18} B + 3 \times 0{,}5\,KiB$

$\cong 3B$

$= 256\,KiB + 1{,}5\,KiB$

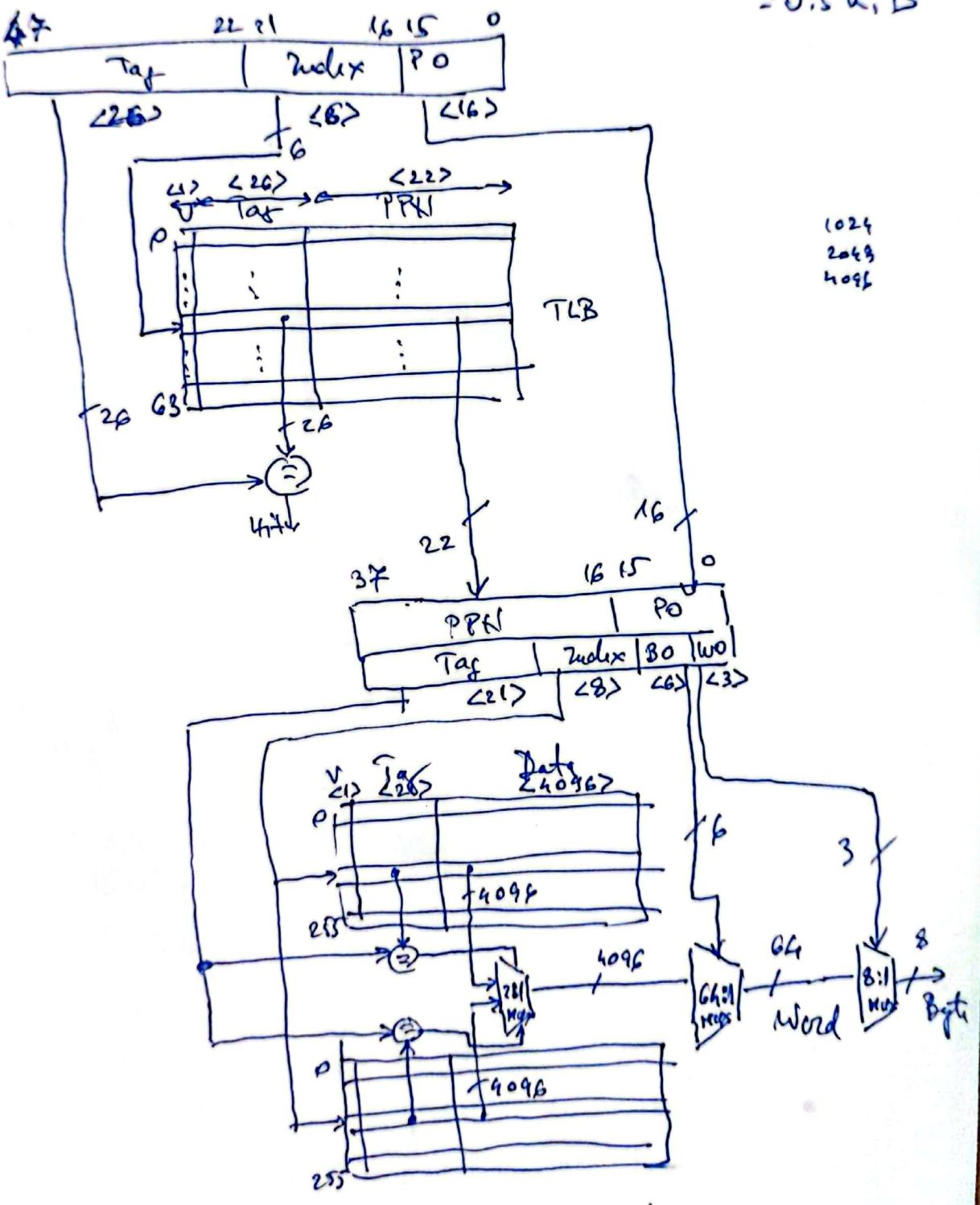
$= 257{,}5\,KiB$

$TLBTSize = 2^6 \times (\underbrace{1}_{V} + \underbrace{26}_{TLBtag} + \underbrace{22}_{PPN})\,bits = 2^6 \times \underbrace{49b}_{\cong 2^6 B} \cong 2^6 \times 2^3 B$

$\cong 2^3 B = 2^9 B$

$= 0.5\,KiB$

c)



Scanned with CamScanner

(4) Se dă un sistem de memorie cu spațiul virtual de dimensiune
(3p) 512 TiB, spațiu fizic de 128 GiB și adresare la nivel de octet. Cuvântul este pe 64 de biți, blocul are 32 de cuvinte. O pagină de memorie are dimensiunea de 16 KiB, iar dimensiunea de date a cache-ului (set 4 căi) este de 128 KiB. Cache-ul este Write Through cu Write No Allocate în caz de Write Miss și are o politică de înlocuire de tip LRU cu age register. TLB-ul are 128 de intrări și folosește mapare set-asociativă pe 2 căi. Se cer:

(1p) b) Dimensiunea totală a memoriei cache și a TLB-ului
(1p) a) Formatul adresei de memorie virtuală, respectiv fizică
(1p) c) Schema bloc a mecanismului adresă virtuală – adresă fizică ce conține memoria cache și TLB-ul (ca în curs pt Fast MATH.)

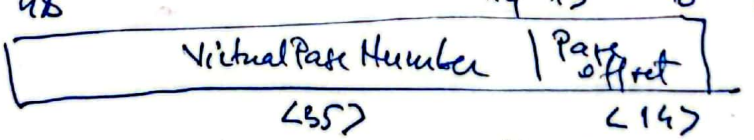
---

a) Virtual Space → 512 TiB = 2^{49} B
Physical Space → 128 GiB = 2^{37} B
1 word = 64 biți = 2^3 B
1 block = 2^5 words = $2^5 \cdot 2^3$ B = 2^8 B

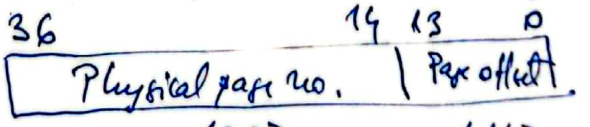
1 page = 2^{14} B
Cache data size = 2^{17} B =
$= \frac{2^{17}\text{ B}}{2^3\text{ B/block}} = 2^9$ blocks
Bank size = $\frac{2^9 \text{ blocks}}{2^2 \text{ s.a}} = 2^7$ blocks

37 − 14 = 23
49 − 14 = 35

Virtual address

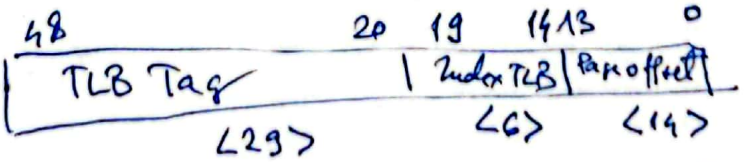


Physical address



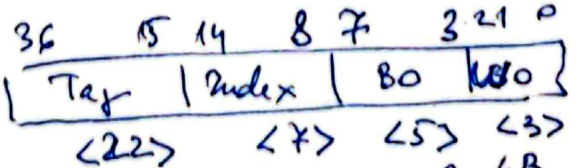
128 entries TLB = 2^7 entries
$\frac{2^7}{2^1 \text{ sa}} = 2^6$ index

Virtual address



✓

Physical address



✓

b) Cache total size = $\underbrace{2^2}_{\text{s.a.}} \times \underbrace{2^7}_{\text{index}} \times (\underbrace{2^8\text{B}}_{\text{data}} + \underbrace{\cong 4\text{B}}_{\substack{1 \\ V}} + \underbrace{2}_{\text{age}} + \underbrace{22}_{\text{tag}}) =$

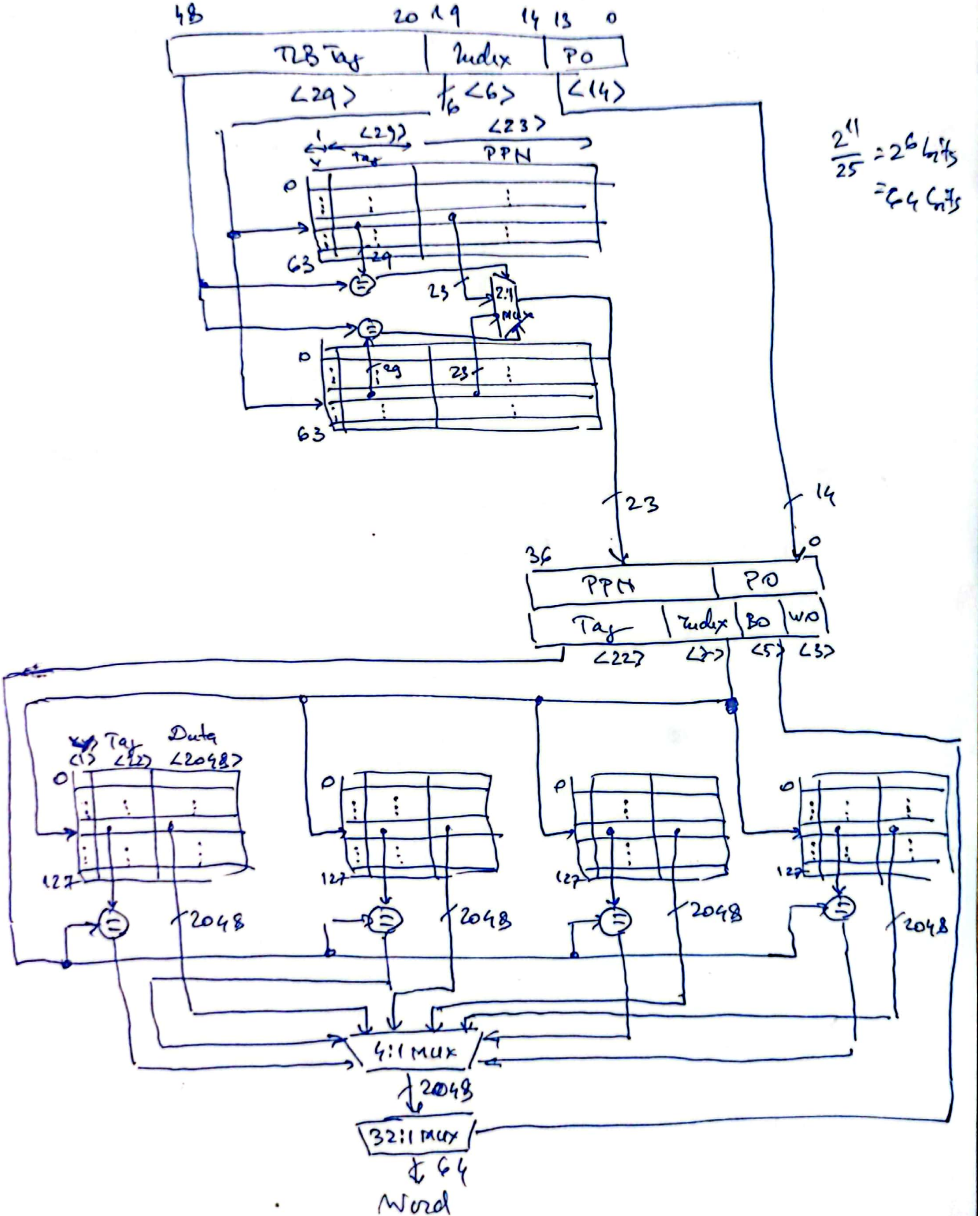
$= 128\text{ KiB} + 2^9 \times 2^2\text{B} = 128\text{ KiB} + 2\text{ KiB} =$ **130 KiB**

Scanned with CamScanner

$TLB_{size} = 2^7 \times (\underbrace{1}_{v} + \underbrace{29}_{tag} + \underbrace{23}_{PPN}) \cong 2^7 (24+32)\,bits =$

$= 2^7 \cdot 7\,B \cong 2^7 \cdot 8\,B \cong \boxed{1\,KiB}$

c)



Scanned with CamScanner